

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

Department of Computer Science and Engineering
Master Degree in Artificial Intelligence

Artificial Intelligence in Industry Project

Forecasting Median House Prices

Luca Corradini Zini

Project Objective

- Predict monthly median house prices using historical property sales data
- Data source: [Kaggle property sales time series dataset](#)
- Understand which factors drive price dynamics over time
- Compare multiple forecasting approaches
- Investigate unexpected model behaviour and improve the feature space

- Data source: Kaggle property sales dataset
- Time span: 2007 – 2019
- Each observation corresponds to a property sale
- Key variables:
 - sale date
 - property price
 - property type (house / unit)
 - number of bedrooms
 - postcode
- Goal: predict a **monthly time series of housing prices**

Project outline

- Problem & dataset
- Extensive EDA
- Data issues and preprocessing (missing values and outliers)
- Feature engineering
- ML models evaluation
- Feature importance analysis
- Observations of results and hypothesis exploration

Dataset

Index	datesold	postcode	price	propertyType	bedrooms
0	2007-02-07	2607	525000	house	4
1	2007-02-27	2906	290000	house	3
2	2007-03-07	2905	328000	house	3
3	2007-03-09	2905	380000	house	4
4	2007-03-21	2906	310000	house	3

- Each row corresponds to a transaction
- Dataset contains thousands of observations
- Time information allows the construction of temporal features

Exploratory Data Analysis

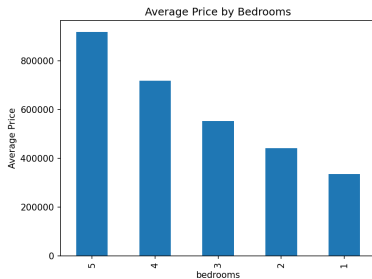
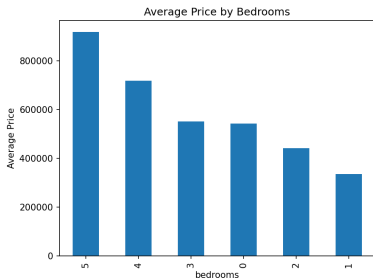
Goals of the EDA:

- Understand distributions
- Identify missing values or anomalies
- Study relationships between features
- Identify potential predictive variables

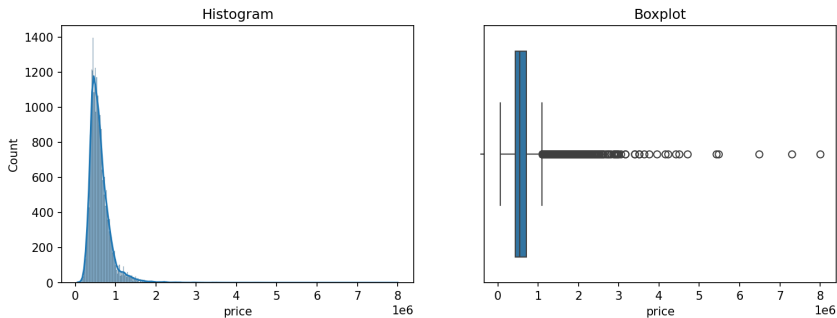
Missing values

The entries corresponding to the number of bedrooms equal to zero were considered as missing values and imputed with KNNImputer.

- KNNImputer uses the average of the nearest neighbors to fill in missing values
- The categorical variables were one-hot encoded before applying KNNImputer ($k=5$)



Distribution of Property Prices



- Prices are strongly right-skewed
- Presence of very expensive properties
- Outliers may affect machine learning models

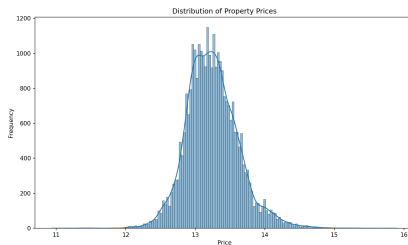
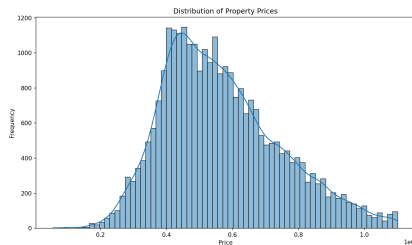
Outlier Handling

I explored different strategies:

- Keep data unchanged
- Remove extreme values using IQR rule:
 - $Q1 = 25^{th}$ percentile, $Q3 = 75^{th}$ percentile, $IQR = Q3 - Q1$
 - Outliers defined as values outside the range:
 $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$
- Apply logarithmic transformation, which is usually effective for right-skewed data

These approaches were tested during model evaluation. Different models preferred different strategies.

Outlier Handling



- Left: Removing outliers using IQR rule
- Right: Logarithmic transformation of prices

From Transactions to Time Series

Individual sales were aggregated into monthly statistics:

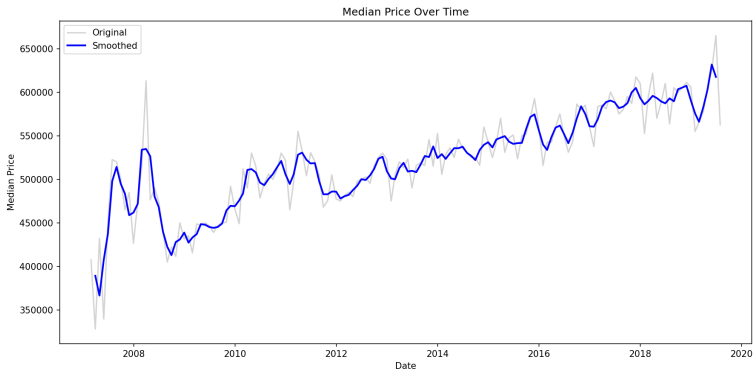
- median price
- number of sales
- average bedrooms
- house ratio (house ratio = $\frac{\text{number of houses sold}}{\text{total sales}}$)

This creates a **structured monthly dataset** suitable for forecasting.

Monthly Trends



Monthly Price Time Series



Observations:

- Clear upward trend in prices over time
- Some seasonal fluctuations visible

Seasonal Decomposition

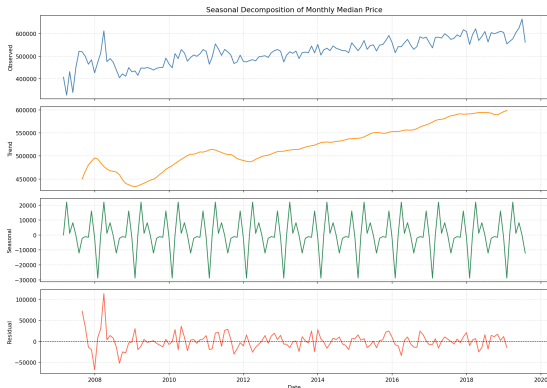
Additive decomposition:

$$y_t = T_t + S_t + R_t$$

- T_t : trend
- S_t : seasonal
- R_t : residual

```
from statsmodels.tsa.seasonal \
    import seasonal_decompose

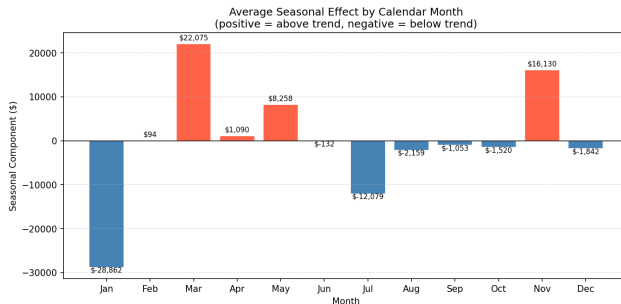
decomp = seasonal_decompose(
    series,
    model='additive',
    period=12
)
```



Seasonal Decomposition

- Trend component shows long-term price increase
- Seasonal component suggests annual cycles
- Residual captures irregular fluctuations. These fluctuations may be influenced by external factors not captured in the dataset (e.g., economic conditions, policy changes).
- The presence of a strong trend and seasonality indicates that models need to account for these patterns to make accurate forecasts.

Seasonal Decomposition: monthly pattern



The seasonal component reveals that certain months consistently show higher or lower prices.

- Prices tend to peak in spring and early summer. This makes sense: many buyers prefer to move during warmer months, and families often want to settle before the new school year.

Additional temporal features were introduced:

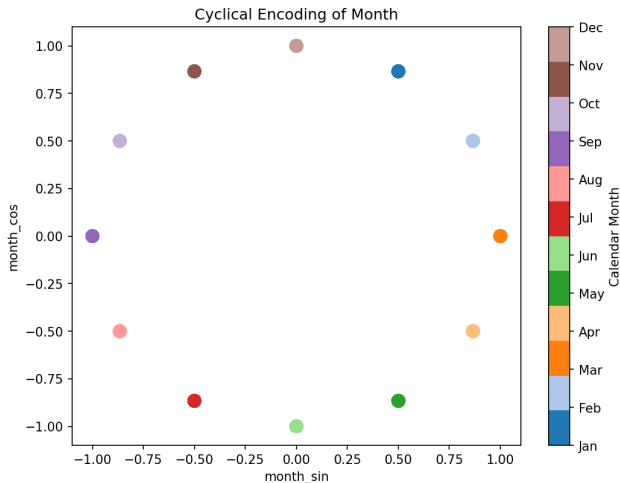
- Month encoded as cyclical variables (in order for December and January to be close in the feature space, instead of 11 months apart):
 - Sin and Cosine transformations of the month variable.

$$x_{\sin} = \sin \left(2\pi \cdot \frac{x}{\max(x)} \right) \quad x_{\cos} = \cos \left(2\pi \cdot \frac{x}{\max(x)} \right)$$

- Scaled year variable representing long-term trend.

This helps machine learning models capture seasonality.

Cyclical Encoding of Months



Cyclical Encoding of Months

- The sine transformation captures the cyclical nature of months, but it is not sufficient to distinguish between all months on its own.
- For example, June and December would have the same sine values (because they are symmetric w.r.t equinoxes)
- By adding the cosine transformation, we can break this symmetry. Each month will have a unique combination of sine and cosine values, allowing the model to differentiate between all months effectively.



Machine Learning Models

Training and Testing Strategy

As it is a time series forecasting problem, the data was split **chronologically**, no random shuffling:

- **Training set:** 2007 – 2017 (about 70% of data, although different splits were tested)
- **Testing set:** 2018 – 2019 (about 30% of data)

Why chronological split?

- A chronological split respects the causal order of events: the model is only ever trained on information that would have been available at prediction time

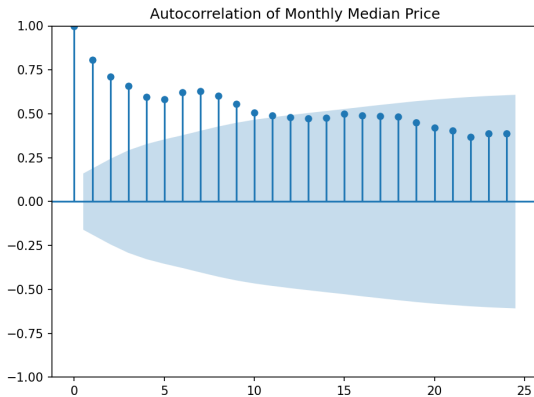
Models were evaluated using:

- Root Mean Squared Error (RMSE): we have a direct measure of how close the predictions are to the actual prices, in dollars.
- Visual inspection of predicted vs actual price trends

Random Forest Regressor

- **Ensemble of decision trees**, each of which is trained on a random subset of the data and features (this reduces overfitting and improves generalization)
- For regression tasks, the final prediction is the average of the predictions from all individual trees. For a single tree, the prediction is made by traversing the tree based on the input features until reaching a leaf node, which contains the predicted value (the average of the target variable for the training samples that fall into that leaf).
- Captures non-linear relationships
- Represents a good baseline because of its interpretability

Random Forest Regressor: Autocorrelation and Lags



The autocorrelation function (ACF) of the monthly price series revealed significant correlations up to lag 12.

Recursive Forecasting with Lagged Features

To leverage temporal dependencies, lagged price features were created. For simplicity, only **lag 1** and **lag 12** were used.

Key question: does using `price_lag_1` at test time cause *data leakage* (i.e. peeking at future true prices)?

Answer: No: the model uses *recursive forecasting*: each prediction is fed back as input for the next step. The true test prices `y_test` are **never accessed** inside the forecasting loop.

Recursive Forecasting: Concrete Example

Setup: 12 months total, last 3 for testing, feature = price_lag_1

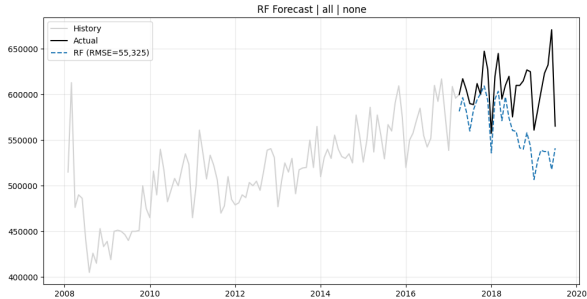
Step	price_lag_1 used	Source
Train: Jan → Sep	true prices	ground truth
Test Oct	p_{Sep} (true)	last real training value ✓
Test Nov	$\hat{p}_{\text{Oct}}$ (predicted)	model's own output ✓
Test Dec	$\hat{p}_{\text{Nov}}$ (predicted)	model's own output ✓

The critical line that prevents leakage:

```
history.append(current_pred)    (never y_test)
```

Trade-off: errors *compound* over time: the further into the test horizon, the more the model relies on its own (potentially noisy) past predictions rather than ground truth.

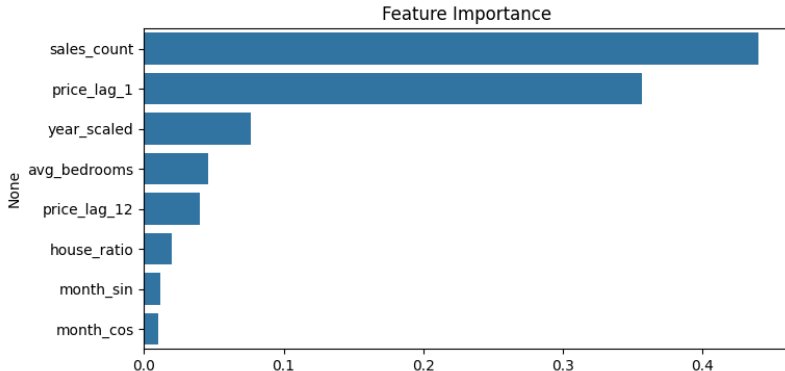
Random Forest Results



Observations:

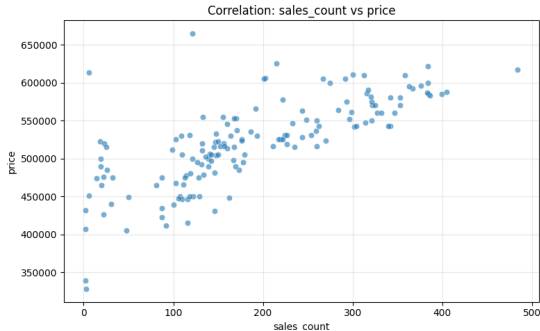
- Model captures overall trend but underestimates recent price growth: **why is that?**

Random Forest Results



Sales volume and the 1-month lagged price are the most important features.

Random Forest Results



There is a strong historical correlation of 0.74. For 85% of our timeline, sales volume was an excellent proxy for demand, and therefore, price.

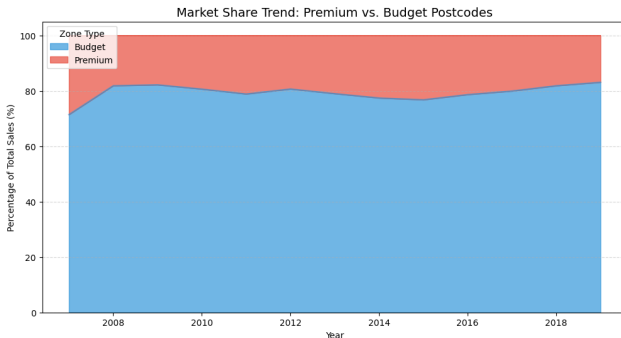
Problem: The model learned a simple rule: lower sales equal a dying market. So, when sales dropped in the final 15% of our data, the model mathematically assumed prices would follow. They didn't.

Hypothesis



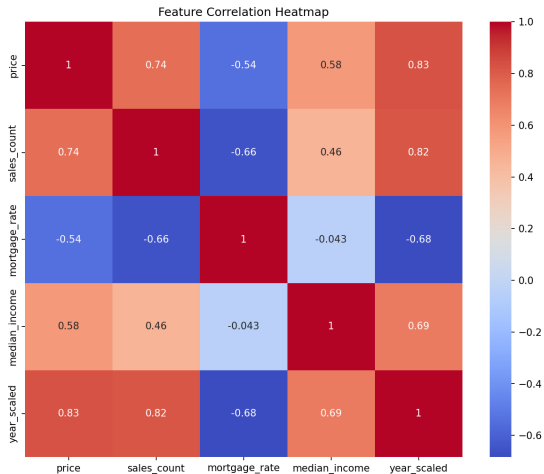
The share of houses (more expensive than units) actually gradually decreased, so this does not explain the price increase.

Hypothesis

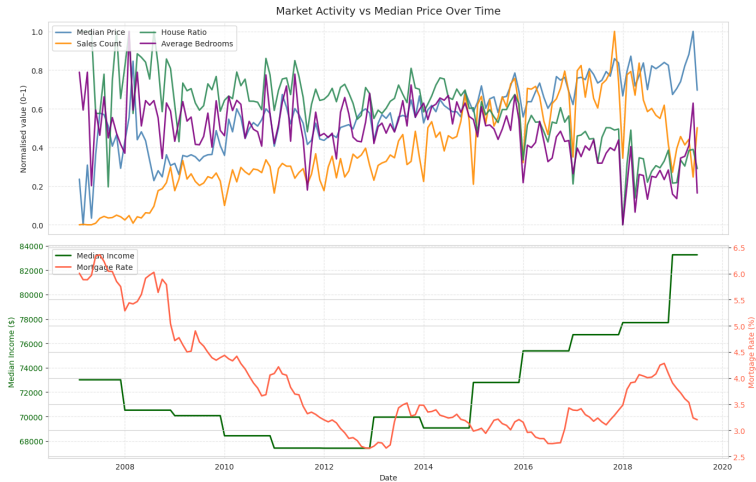


The share of sales in top-tier zip codes remained relatively stable.
We are not just seeing a luxury market distortion.

Two new features



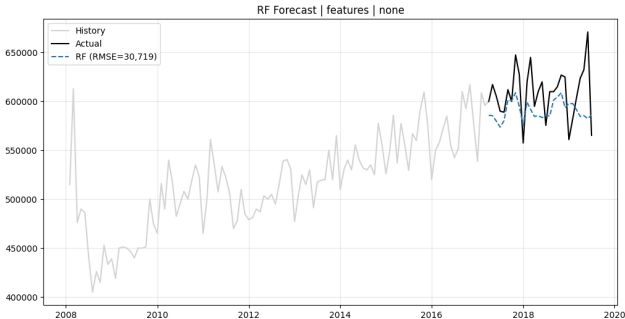
Random Forest Results



By looking at the trends in later years (where the apparent inconsistency occurs), we can see that:

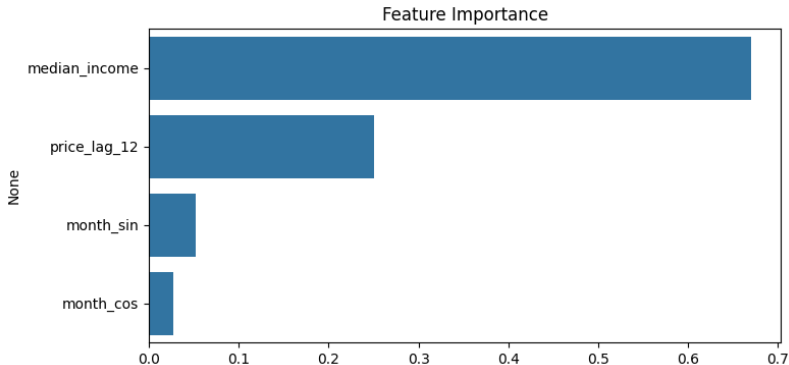
- Mortgage (negatively correlated with sales and prices) first sharply increased, then decreased. This could have caused a temporary drop in sales, but also a subsequent boost in prices as affordability improved.
- Rising family income can also boost prices (demand and supply), even if fewer transactions occur.
- These macro factors can explain why prices continued to rise despite declining sales in the later years of our dataset.

Random Forest Results



After testing all combinations of features, the best performance was achieved by including macroeconomic variables and excluding sales volume. The model now captures the price increase in later years much better.

Random Forest Results



Family income is now among the most important features, while sales volume is no longer used.

A better solution: hybrid model

- This dataset exhibits a strong **linear growing trend**, a **seasonal component**, and a **residual component** driven by other factors.

Step 1: Ridge Regression

- Captures the overall long-term trend

+

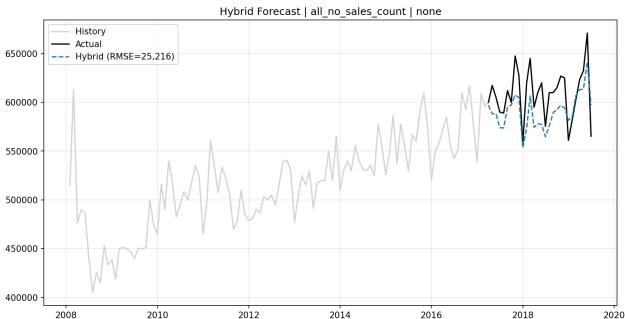
Step 2: Random Forest

- Fits the residuals to capture non-linear patterns and seasonality

Combined Prediction

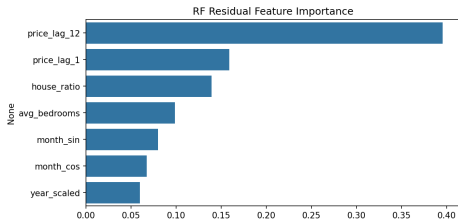
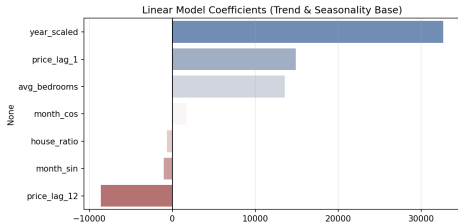
$$\text{Residual} = y - \hat{y}_{\text{Ridge}} \quad \Rightarrow \quad \hat{y} = \hat{y}_{\text{Ridge}} + \hat{y}_{\text{RF}}$$

Hybrid Model Results



- RMSE is reduced of 17% compared to the best random forest model, and the price increase in later years is now fully captured.
- The 'ups and downs' of the price trend are now much better represented, thanks to the random forest capturing the residuals.

Hybrid Model Results



- The feature 'year' (as well as the lagged price and the number of bedrooms) is strongly linearly correlated with price, while mortgage rates are negatively correlated. Exactly as expected, and as seen in the correlation matrix.
- On the other hand, the house to unit ratio, and the cyclical month features are more important for the random forest, which captures the non-linear and seasonal patterns.

Deep Learning Approach: LSTM Networks

Why LSTMs are theoretically a strong solution

Long Short-Term Memory (LSTM) networks are designed specifically for sequential data:

- Through their *Cell State*, they can theoretically maintain a long-term "macro trend" while the *hidden state* captures short-term "ups and downs".
- **The Forget Gate Mechanism:** This allows the network to adaptively decide which historical shocks are noise and which are structural shifts.
- **Non-Linearity:** Unlike traditional models, LSTMs can learn highly complex, non-linear interactions between features like interest rates and price without explicit manual engineering.

Empirical Findings: The problem of data scarcity

Observations from implementation

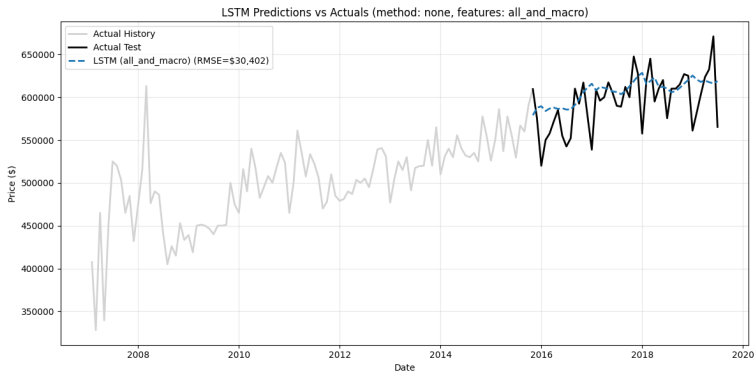
While LSTMs are powerful in big data contexts, our specific dataset had important limitations:

- **High Variance & Unreliability:** Due to the small sample size (≈ 105 months in the training set), the model was extremely sensitive to random weight initialization.
- **Lack of Reproducibility:** Identical training runs yielded significantly different RMSE scores and forecast trajectories.
- **The Smoothing Effect:** The architecture naturally tended to "filter out" the high-frequency fluctuations (wiggles) as noise, resulting in a smoothed average rather than a reactive forecast.

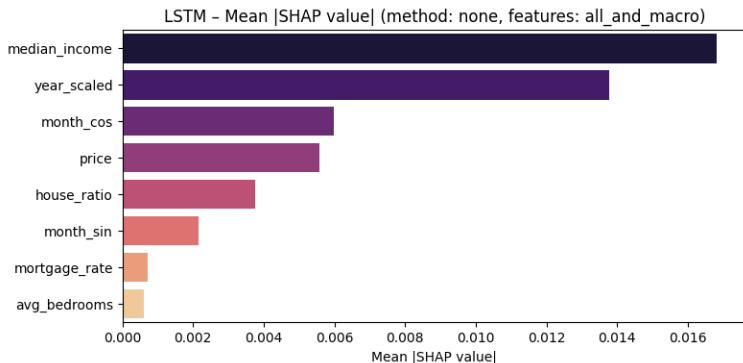
Conclusion

The model proved too **stochastic** and **unstable** for this volume of data.

LSTM Results: Forecast vs Actual



LSTM Results: SHAP Feature Importance



What is SARIMAX?

SARIMAX = Seasonal AutoRegressive Integrated Moving Average with eXogenous variables.

- Time series forecasting statistical model (can be used with limited data, it is not a data-hungry deep learning model)
- Combines regression and autoregressive dynamics
- Captures:
 - temporal dependence
 - seasonality
 - external predictors

Model structure:

$$y_t = AR + MA + Seasonal + Exogenous + \epsilon$$

Handling Non-Stationary Data

Time series often contain trends:

- Housing prices increase over time
- Data is therefore non-stationary

SARIMAX solves this using **differencing**:

$$y'_t = y_t - y_{t-1}$$

This removes the trend and makes the series stationary.

The model then learns patterns in the stationary series. Then, predictions are transformed back to the original scale by reversing the differencing:

$$\hat{y}_t = \hat{y}'_t + y_{t-1}$$

Model Structure

Selected model:

$$\text{SARIMAX}\left(\underbrace{\underbrace{2}_{\text{AR}}, \underbrace{1}_{\text{I}}, \underbrace{0}_{\text{MA}}}_{\text{non-seasonal}} \times \left(\underbrace{\underbrace{1}_{\text{SAR}}, \underbrace{0}_{\text{SI}}, \underbrace{1}_{\text{SMA}}}_{\text{seasonal}} \underbrace{12}_{\text{period}}\right)\right)$$

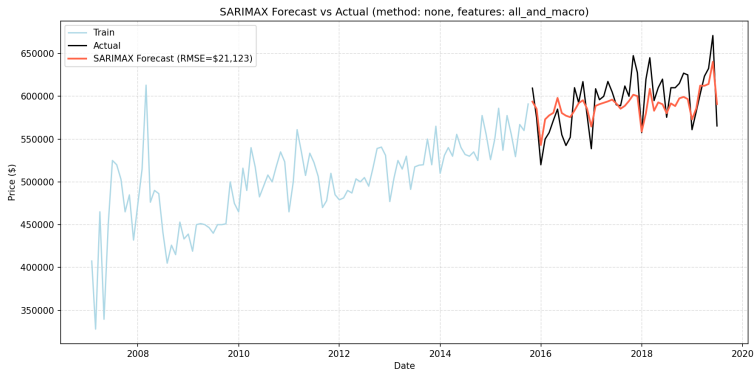
Meaning:

- AR(2): uses two previous observations (lags 1 and 2)
- I(1): first difference applied to achieve stationarity
- MA(0): no moving average component (past errors not used)
- Seasonal AR(1): yearly dependence
- Seasonal differencing not needed (seasonality is stable)
- Seasonal MA(1): seasonal error correction
- Period = 12 months

How does SARIMAX fit the data?

- The model identifies the best combination of AR, I, MA, and seasonal parameters by minimizing the AIC (Akaike Information Criterion).
 - AIC balances model fit and complexity, penalizing overfitting.
- It captures both short-term dependencies (AR and MA) and long-term seasonal patterns (seasonal AR and MA).
- Exogenous variables can be included to account for external factors influencing prices. In our case, we included the features of the dataset.

SARIMAX Results: Forecast vs Actual



MAPE (Mean Absolute Percentage Error) is 3.06%, which is a very good result.

Conclusion

- Machine learning models can effectively forecast housing prices
- Understanding model errors can reveal deeper insights
- Integrating macroeconomic indicators significantly improves interpretation

Future work:

- incorporate additional economic indicators
- explore spatial models
- test alternative deep learning architectures

References

- Kaggle Housing Sales Dataset
- Housing Market Forecasting Literature
- Time Series Forecasting with Decomposition